

Name: Monisha Raghini Chelle

RollNo: 2520030537

Section - 6

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

17. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSLAB14  Simple-Commands-Week1  foldername  hello.c  monis  sample1.c
OSLAB                           OSSP-Practical  example.txt             hello       moni     sample1  sample1.c.save
monis@DESKTOP-C3GM4T6:~$ cd OSLAB
monis@DESKTOP-C3GM4T6:~/OSLAB$ ls
OSLAB28  Scheduling-Algorithms  hello  hello.c:Zone.Identifier  hello_link  newfile.txt  test1
Pipes    file1.txt              hello.c  hello:Zone.Identifier    hello_shortcut  skill        virtual_address
monis@DESKTOP-C3GM4T6:~/OSLAB$ cd skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 2 — Create list_jobs.c

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano list_jobs.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano list_jobs.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra list_jobs.c -o list_jobs
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 list_jobs.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef struct {
    int id;
    pid_t pid;
    int active;
    char command[100];
} Job;

Job jobs[MAX_JOBS];
int next_id = 1;

void update_jobs(void)
{
    int status;

    for (int i = 0; i < MAX_JOBS; i++) {
        if (!jobs[i].active)
            continue;

        pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

        if (result == jobs[i].pid) {
            jobs[i].active = 0;
            printf("Job [%d] completed.\n", jobs[i].id);
        }
    }
}

void list_jobs(void)
{
    int found = 0;

    update_jobs();

    printf("\nJOB ID\tPID\tSTATUS\tCOMMAND\n");

    for (int i = 0; i < MAX_JOBS; i++) {
        if (jobs[i].active) {
            printf("[%d]\t%d\tRUNNING\t%s\n",
                jobs[i].id,
                (long)jobs[i].pid,
                jobs[i].command);
            found = 1;
        }
    }

    if (!found)
        printf("No active jobs.\n");
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 list_jobs.c *
}
void start_job(int seconds)
{
    int slot = -1;

    for (int i = 0; i < MAX_JOBS; i++) {
        if (!jobs[i].active) {
            slot = i;
            break;
        }
    }

    if (slot == -1) {
        printf("Job table full.\n");
        return;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return;
    }

    if (pid == 0) {
        char duration[20];
        snprintf(duration, sizeof(duration), "%d", seconds);
        execlp("sleep", "sleep", duration, (char *)NULL);
        perror("execlp");
        _exit(127);
    }

    jobs[slot].id = next_id++;
    jobs[slot].pid = pid;
    jobs[slot].active = 1;

    snprintf(jobs[slot].command,
             sizeof(jobs[slot].command),
             "sleep %d", seconds);

    printf("Started job [%d], PID=%ld\n",
           jobs[slot].id, (long)pid);
}

int main(void)
{
    char input[100];
    int seconds;
    char extra;

    printf("Q17 - List Active Jobs\n");
    printf("Commands: add NUMBER, jobs, exit\n");

    while (1) {
        update_jobs();
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 list_jobs.c *
jobs[slot].id = next_id++;
jobs[slot].pid = pid;
jobs[slot].active = 1;

snprintf(jobs[slot].command,
        sizeof(jobs[slot].command),
        "sleep %d", seconds);

printf("Started job [%d], PID=%ld\n",
        jobs[slot].id, (long)pid);
}

int main(void)
{
    char input[100];
    int seconds;
    char extra;

    printf("Q17 - List Active Jobs\n");
    printf("Commands: add NUMBER, jobs, exit\n");

    while (1) {
        update_jobs();

        printf("\njoblist> ");
        fflush(stdout);

        if (!fgets(input, sizeof(input), stdin))
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strcmp(input, "jobs") == 0) {
            list_jobs();
        }
        else if (sscanf(input, "add %d %c",
                        &seconds, &extra) == 1 &&
                seconds >= 0) {
            start_job(seconds);
        }
        else {
            printf("Invalid command.\n");
        }
    }

    for (int i = 0; i < MAX_JOBS; i++) {
        if (jobs[i].active)
            waitpid(jobs[i].pid, NULL, 0);
    }

    printf("Job listing program exited.\n");
    return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-F Redo

Save using Ctrl + O, Enter, then Ctrl + X.

```
Write to File: list_jobs.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse  
^C Cancel    M-M Mac Format  M-P Prepend    ^Q Discard buffer
```

Compile:

gcc -Wall -Wextra list_jobs.c -o list_jobs

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano list_jobs.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra list_jobs.c -o list_jobs
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 3 — Test Active Job Listings

Run:

`./list_jobs`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano list_jobs.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra list_jobs.c -o list_jobs
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./list_jobs
Q17 - List Active Jobs
Commands: add NUMBER, jobs, exit

joblist>
```

Enter these commands one at a time:

`add 60`

`add 90`

`jobs`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./list_jobs
Q17 - List Active Jobs
Commands: add NUMBER, jobs, exit

joblist> add 60
Started job [1], PID=658

joblist> add 90
Started job [2], PID=659

joblist> jobs
JOB ID  PID    STATUS  COMMAND
[1]     658    RUNNING sleep 60
[2]     659    RUNNING sleep 90

joblist>
```

Step 4 — Create foreground_switch.c

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano foreground_switch.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 foreground_switch.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/wait.h>

static pid_t shell_pgid;
static int next_job_id = 1;

typedef struct {
    int id;
    pid_t pid;
    pid_t pgid;
    int active;
    int stopped;
} Job;

static Job job = {0};

void update_job(void)
{
    int status;

    if (!job.active)
        return;

    pid_t result = waitpid(
        job.pid, &status,
        WNOHANG | WUNTRACED | WCONTINUED
    );

    if (result == job.pid) {
        if (WIFEXITED(status) || WIFSIGNALED(status)) {
            printf("\nJob [%d] completed.\n", job.id);
            job.active = 0;
            job.stopped = 0;
        }
        else if (WIFSTOPPED(status)) {
            job.stopped = 1;
        }
        else if (WIFCONTINUED(status)) {
            job.stopped = 0;
        }
    }
}

void start_job(int seconds)
{
    if (job.active) {
        printf("Only one active job is supported.\n");
        return;
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 foreground_switch.c *

pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    return;
}

if (pid == 0) {
    setpgid(0, 0);

    signal(SIGINT, SIG_DFL);
    signal(SIGTSTP, SIG_DFL);
    signal(SIGTTIN, SIG_DFL);
    signal(SIGTTOU, SIG_DFL);

    char duration[20];
    sprintf(duration, "%d", seconds);

    execlp("sleep", "sleep", duration, (char *)NULL);
    perror("execlp");
    _exit(127);
}

if (setpgid(pid, pid) == -1 && errno != EACCES)
    perror("setpgid");

job.id = next_job_id++;
job.pid = pid;
job.pgid = pid;
job.active = 1;
job.stopped = 0;

printf("Started background job [%d], PID=%ld, PGID=%ld\n",
       job.id, (long)job.pid, (long)job.pgid);
}

void show_job(void)
{
    update_job();

    if (!job.active) {
        printf("No active job.\n");
        return;
    }

    printf("[%d] PID=%ld PGID=%ld STATUS=%s\n",
           job.id,
           (long)job.pid,
           (long)job.pgid,
           job.stopped ? "STOPPED" : "RUNNING");
}

void foreground_job(int id)
{
    int status;
```

Help	Write Out	Where Is	Cut	Execute	Location	Undo
Exit	Read File	Replace	Paste	Justify	Go To Line	Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 foreground_switch.c *

update_job();

if (!job.active || job.id != id) {
    printf("Job not found.\n");
    return;
}

printf("Moving job [%d] to foreground...\n", job.id);
fflush(stdout);

if (tcsetpgrp(STDIN_FILENO, job.pgid) == -1) {
    perror("tcsetpgrp");
    return;
}

if (job.stopped) {
    if (kill(-job.pgid, SIGCONT) == -1) {
        perror("SIGCONT");
        tcsetpgrp(STDIN_FILENO, shell_pgid);
        return;
    }

    job.stopped = 0;
    printf("Job resumed.\n");
    fflush(stdout);
}

pid_t result;

do {
    result = waitpid(job.pid, &status, WUNTRACED);
} while (result == -1 && errno == EINTR);

/* Return terminal control to this program. */
if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
    perror("restore terminal");

if (result == -1) {
    perror("waitpid");
    return;
}

if (WIFSTOPPED(status)) {
    job.stopped = 1;
    printf("Job [%d] stopped.\n", job.id);
}
else {
    printf("Job [%d] completed in foreground.\n", job.id);
    job.active = 0;
    job.stopped = 0;
}
}

int main(void)
{
    char input[100];

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 foreground_switch.c *
char input[100];
int number;
char extra;

if (!isatty(STDIN_FILENO)) {
    fprintf(stderr, "Run in an interactive terminal.\n");
    return 1;
}

shell_pgid = getpgrp();

signal(SIGTTOU, SIG_IGN);
signal(SIGTTIN, SIG_IGN);

printf("Q17 - Foreground Switching\n");
printf("Commands: add NUMBER, jobs, fg JOB_ID, exit\n");

while (1) {
    update_job();

    printf("\nfgshell> ");
    fflush(stdout);

    if (!fgets(input, sizeof(input), stdin))
        break;

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "exit") == 0)
        break;

    if (strcmp(input, "jobs") == 0) {
        show_job();
    }
    else if (sscanf(input, "add %d %c",
                    &number, &extra) == 1 &&
             number >= 0) {
        start_job(number);
    }
    else if (sscanf(input, "fg %d %c",
                    &number, &extra) == 1) {
        foreground_job(number);
    }
    else {
        printf("Invalid command.\n");
    }
}

if (job.active) {
    if (job.stopped)
        kill(-job.pgid, SIGCONT);

    waitpid(job.pid, NULL, 0);
}

printf("Foreground switching program exited.\n");
return 0;

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 foreground_switch.c *
char extra;

if (!isatty(STDIN_FILENO)) {
    fprintf(stderr, "Run in an interactive terminal.\n");
    return 1;
}

shell_pgid = getpgid();

signal(SIGTTOU, SIG_IGN);
signal(SIGTTIN, SIG_IGN);

printf("Q17 - Foreground Switching\n");
printf("Commands: add NUMBER, jobs, fg JOB_ID, exit\n");

while (1) {
    update_job();

    printf("\nfgshell> ");
    fflush(stdout);

    if (!fgets(input, sizeof(input), stdin))
        break;

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "exit") == 0)
        break;

    if (strcmp(input, "jobs") == 0) {
        show_job();
    }
    else if (sscanf(input, "add %d %c",
                    &number, &extra) == 1 &&
             number >= 0) {
        start_job(number);
    }
    else if (sscanf(input, "fg %d %c",
                    &number, &extra) == 1) {
        foreground_job(number);
    }
    else {
        printf("Invalid command.\n");
    }
}

if (job.active) {
    if (job.stopped)
        kill(-job.pgid, SIGCONT);

    waitpid(job.pid, NULL, 0);
}

printf("Foreground switching program exited.\n");
return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Save using Ctrl + O, Enter, then Ctrl + X.

```
return 0;
}

Write to File: foreground_switch.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse  
^C Cancel     M-M Mac Format  M-P Prepend    ^Q Discard buffer
```

Save and compile:

gcc -Wall -Wextra foreground_switch.c -o foreground_switch

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano foreground_switch.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra foreground_switch.c -o foreground_switch
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 5 — Test Foreground Switching

Run:

`./foreground_switch`

Enter:

add 30

Enter:

jobs

Now enter:

fg 1

Enter jobs again:

No active job.

Then enter exit.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./foreground_switch
Q17 - Foreground Switching
Commands: add NUMBER, jobs, fg JOB_ID, exit

fgshell> add 30
Started background job [1], PID=750, PGID=750

fgshell> jobs
[1] PID=750 PGID=750 STATUS=RUNNING

fgshell> fg 1
Moving job [1] to foreground...
Job [1] completed in foreground.

fgshell> No active job
Invalid command.

fgshell>
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./foreground_switch
Q17 - Foreground Switching
Commands: add NUMBER, jobs, fg JOB_ID, exit

fgshell> add 30
Started background job [1], PID=750, PGID=750

fgshell> jobs
[1] PID=750 PGID=750 STATUS=RUNNING

fgshell> fg 1
Moving job [1] to foreground...
Job [1] completed in foreground.

fgshell> No active job
Invalid command.

fgshell> exit
Foreground switching program exited.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

ls -l *.c list_jobs foreground_switch

```
fgshell> exit
Foreground switching program exited.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l *.c list_jobs foreground_switch
-rw-r--r-- 1 monis monis 2374 Sep 18 12:29 append_redirection.c
-rw-r--r-- 1 monis monis 5930 Sep 25 18:56 background_jobs.c
-rw-r--r-- 1 monis monis 2171 Sep 17 22:50 builtin_dispatch.c
-rw-r--r-- 1 monis monis 1992 Sep 17 22:20 bultins.c
-rw-r--r-- 1 monis monis 590 Sep 17 19:56 child_process.c
-rw-r--r-- 1 monis monis 3190 Sep 25 10:57 combined_redirection.c
-rw-r--r-- 1 monis monis 2401 Sep 18 10:28 command_history.c
-rw-r--r-- 1 monis monis 630 Sep 17 20:17 command_launch.c
-rw-r--r-- 1 monis monis 6003 Sep 25 11:45 complex_commands.c
-rw-r--r-- 1 monis monis 6421 Sep 25 11:53 complex_commands1.c
-rw-r--r-- 1 monis monis 2535 Sep 17 22:39 directory_navigation.c
-rw-r--r-- 1 monis monis 1320 Sep 17 23:07 directory_state.c
-rw-r--r-- 1 monis monis 2175 Sep 17 21:29 double_quotes.c
-rw-r--r-- 1 monis monis 2443 Sep 17 19:35 escape_parser.c
-rw-r--r-- 1 monis monis 733 Sep 17 20:00 execution_error.c
-rw-r--r-- 1 monis monis 588 Sep 17 23:32 export_child.c
-rw-r--r-- 1 monis monis 1969 Sep 17 23:31 export_variables.c
-rwxr-xr-x 1 monis monis 17256 Sep 26 21:42 foreground_switch
-rw-r--r-- 1 monis monis 4640 Sep 26 21:41 foreground_switch.c
-rw-r--r-- 1 monis monis 3804 Sep 16 08:42 history.c
-rw-r--r-- 1 monis monis 2067 Sep 18 11:32 input_redirection.c
-rw-r--r-- 1 monis monis 6547 Sep 25 19:09 job_table.c
-rwxr-xr-x 1 monis monis 16768 Sep 26 21:36 list_jobs
-rw-r--r-- 1 monis monis 2837 Sep 26 21:35 list_jobs.c
-rw-r--r-- 1 monis monis 2124 Sep 18 11:20 long_pipeline.c
-rw-r--r-- 1 monis monis 316 Sep 16 09:27 memory.c
-rw-r--r-- 1 monis monis 846 Sep 16 09:32 memory1.c
-rw-r--r-- 1 monis monis 1002 Sep 16 09:35 memory2.c
-rw-r--r-- 1 monis monis 861 Sep 16 09:39 memory3.c
-rw-r--r-- 1 monis monis 1165 Sep 16 09:46 memory4.c
-rw-r--r-- 1 monis monis 1789 Sep 18 11:14 multiple_pipes.c
-rw-r--r-- 1 monis monis 2379 Sep 18 11:48 output_redirection.c
-rw-r--r-- 1 monis monis 779 Sep 17 20:11 parent_child.c
-rw-r--r-- 1 monis monis 3355 Sep 17 21:51 parser_logic.c
-rw-r--r-- 1 monis monis 677 Sep 17 20:05 pass_arguments.c
-rw-r--r-- 1 monis monis 1073 Sep 17 21:01 path_missing.c
-rw-r--r-- 1 monis monis 1479 Sep 17 20:55 path_resolution.c
-rw-r--r-- 1 monis monis 1114 Sep 18 11:06 pipe_sync.c
-rw-r--r-- 1 monis monis 2309 Sep 18 10:40 pipeline_structure.c
-rw-r--r-- 1 monis monis 894 Aug 24 21:33 process.c
-rw-r--r-- 1 monis monis 907 Sep 18 10:48 process_pipeline.c
-rw-r--r-- 1 monis monis 1789 Aug 24 22:19 shell.c
-rw-r--r-- 1 monis monis 796 Sep 18 11:01 single_pipe.c
-rw-r--r-- 1 monis monis 897 Sep 17 21:19 single_quotes.c
-rw-r--r-- 1 monis monis 2726 Sep 18 13:53 stderr_redirection.c
-rw-r--r-- 1 monis monis 2034 Sep 16 15:24 tokenization.c
-rw-r--r-- 1 monis monis 2034 Sep 16 15:20 tokeniztion.c
-rw-r--r-- 1 monis monis 1579 Sep 17 22:08 variable_expansion.c
-rw-r--r-- 1 monis monis 871 Sep 17 20:45 waitpid_monitor.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```